

When a file is compiled, GCC generates machine code and stores it in an **ELF (Executable and Linkable Format)** file. ELF is used for relocatable object files (`.o`), executables, and shared libraries (`.so`).

The source code is compiled into machine code, and additional metadata is added for **linking, loading, and execution**.

An ELF file contains several main parts:

- **ELF Header:** Contains general information about the file, such as its type, architecture, entry point, and offsets to other ELF structures.
- **Program Headers:** Describe how parts of the file should be loaded into memory. They are used by executables and shared libraries, but normally do not exist in relocatable object files (`.o`).
- **Section Headers:** Contain metadata about each section, such as its type, size, address, and file offset. They are mainly used during linking and by tools such as `nm`. A stripped binary may have its symbol information removed to reduce its size.
- **Sections:** Contain the actual data, such as machine code (`.text`), read-only data (`.rodata`), initialized data (`.data`), uninitialized data (`.bss`), symbol tables, and string tables.

How `nm` reads an ELF file

When `nm` is executed, it reads the ELF header to determine the file structure. It then uses the **section header table** to locate the symbol table (`.symtab` or `.dynsym`) and its associated string table. It parses the symbols and displays their names, addresses, and types.

What happens during execution

When an executable starts, the **kernel** loads the required parts of the ELF file into virtual memory. For dynamically linked programs, the **dynamic linker/loader** (such as `ld-linux`) loads the required shared libraries and performs dynamic symbol resolution and relocation before execution continues.